2장

데이터 입력하고 결과 출력하기

1장에서는 프로그램과 프로그래밍이 무엇인지, 파이썬이 무엇인지 살펴보았습니다. 또한, 파이썬으로 프로그램을 작성하기 위한 준비까지 마쳤습니다. 이 장에서는 앞서 배웠던 명령어의 기본 형식을 배우고 그 다음으로 가장 기본 명령어인 입력과 출력 명령어를 알아보겠습니다.

2.1

파이썬 명령어의 형식

사람의 언어에도 수식구조, 서술구조와 같은 문법구조가 있듯이 프로그래밍 언어에도 형식이 있습니다. 파이썬에서는 다음과 같이 명령 내용을 뜻하는 영단어 뒤에 소괄호(`()`)를 붙입니다.

```
print()
input()
list()
next()
```

명령어와 각 명령어에 해당하는 동작 내용은 미리 정해져 있습니다. 대부분 명령어를 이루는 영단어의 뜻과 유사합니다.

`print()`는 print 단어의 뜻처럼 '출력하라'는 명령어고, `input()`은 input의 뜻처럼 입력과 관련한 일을 컴퓨터가 하게 하는 명령어입니다. 같은 이유로 나머지 명령어의 의미를 추측해 보면 `list()`는 '리스트로 만들어라'이고, `next()`는 '다음으로 가라'는 명령어일 겁니다(`list()`는 6장에서, `next()`는 9장에서 배웁니다).

- 명령어 형식: 영단어( )
- 명령어 의미: 컴퓨터야, 미리 정해진 명령 내용대로 일을 해라!

명령어에 사용하는 영단어는 아무거나 쓸 수 없습니다. 해당 프로그래밍 언어를 만든 사람이 사용할 수 있는 단어를 미리 정해 놓았기 때문입니다. 예를 들어 memorize는 미리 정해 놓은 단어가 아니어서 `memorize()`라고 명령해도 컴퓨터가 알아듣지 못합니다. 따라서 프로그래밍을 처음 배울 때는 미리 정해 놓은(정의된) 명령어에 어떤 것이 있는지를 파악해야 합니다. 이 책에서는 파이썬의 기본 명령어를 익히고, 더 나아가 나만의 명령어를 만들 수 있는 사용자 정의 함수까지 배우겠습니다.

1분 퀴즈

1 다음 중 틀린 것을 고르세요.

① 프로그래밍은 컴퓨터가 사람의 의도대로 일을 수행하는 프로그램을 만드는 일이다.

② 사람이 시키는 대로 컴퓨터가 동작하게 하려면 명령어를 사용해야 한다.

③ 명령어는 특별한 형식이 없고 `print`, `input`과 같은 영단어만 쓰면 된다.

④ 컴퓨터가 알아듣는 명령어는 미리 정해져 있다.

정답 및 해설

2.2

실행결과를 보여 주는 출력 명령어

명령어 형식을 알았으니 파이썬 명령어를 하나씩 배워 보겠습니다. 첫 번째로 배울 명령어는 기본 중의 기본으로, 모든 프로그래밍 언어에서 절대 빠질 수 없는 출력 명령어입니다. 출력 명령어를 배우기 전에 '출력'의 정확한 뜻은 무엇인지부터 살펴볼까요?

2.2.1 출력이란

우리는 이미 수많은 출력을 접하고 있습니다. 포털 사이트에서 검색창에 검색어를 입력했을 때 나오는 뉴스, 블로그, 쇼핑 정보 등은 모두 출력 명령어의 결과물입니다.

메신저 프로그램에서 친구가 보낸 메시지가 화면에 나오고, 내가 보내는 메시지도 전송 버튼을 누르면 화면에 표시됩니다. 스마트폰의 날씨 앱은 사용자가 지정한 지역의 날씨를 간단한 아이콘으로 보여 줍니다. 이처럼 컴퓨터가 사람(프로그램 사용자)이 내린 명령을 실행한 결과를 기기(모니터나 스마트폰 등)의 화면을 통해 보여 주는 것을 출력이라고 합니다.

그림 2-1 출력



▲ 그림 2-1 출력

우리가 프로그램을 사용하는 이유는 프로그램이 가진 고유의 기능을 활용해 결과를 빠르게 확인하기 위해서입니다. 날씨 앱(프로그램)으로는 오늘의 날씨(결과)를 확인하고, 검색 사이트(프로그램)에서는 검색어와 관련한 정보(결과)를 얻습니다. 메신저(프로그램)로는 물리적으로 떨어져 있는 사람과 실시간으로 대화(결과)합니다. 그런데 프로그램이 사용자에게 실행결과를 보여 주지 못하면 프로그램을 사용하는 의미가 없겠죠? 그래서 출력은 간단하고 당연해 보이지만, 프로그래밍에서 매우 중요한 요소입니다.

그럼 출력 명령어를 어떻게 사용하는지 배워 봅시다.

2.2.2 출력 명령어 print( )

파이썬에서 출력 명령어는 `print()`입니다. 명령어 형식대로 출력을 뜻하는 영단어 print에 소괄호가 붙습니다. 소괄호 안에 다양한 것을 넣을 수 있으며, 소괄호 안 내용을 실행한 결과를 출력합니다.

```
print(): 출력해라.
```

이 명령어가 어떤 일을 하는지 한번 볼까요? 1장에서 설명한 대로 코랩 또는 주피터 노트북 파일 하나를 만듭니다. 파일명은 편한 대로 정하세요. 첫 번째 셀에 다음과 같이 코드를 작성하고 실행합니다.

```
print(123)
```

```
실행결과
123
```

괄호 안에 넣은 123이 결과창에 출력됩니다. 이렇듯 `print()` 명령어는 괄호 안에 넣은 내용을 결과로 출력합니다. 다른 숫자도 넣어 볼까요? 다음 셀에 코드를 작성하세요.

```
print(2020)
```

```
실행결과
2020
```

예상한 대로 2020이 출력되네요. 이번에는 문자를 넣어 봅시다.

```
print(가나다)
```

```
실행결과
----> 1 print(가나다)
NameError: name '가나다' is not defined
```

'가나다'가 아니라 이상한 메시지가 나옵니다. 뭔가 잘못된 것 같죠? 이렇게 원하거나 예상한 결과가 아닌 어떤 문제 상황이 발생할 때 이를 오류(error)라고 합니다. 오류는 3장에서 설명하니 여기서는 오류가 있다는 것만 알고 넘어갑시다.

숫자는 `print()` 명령어에 넣은 그대로 나왔는데, 문자는 오류가 발생하네요. 이는 컴퓨터가 단순해서 그렇습니다. 컴퓨터는 숫자와 문자를 구분하지 못합니다. 하나만 알고 둘은 모르죠. 문자를 넣을 때는 '이것은 문자다'라고 표시해 줘야 알아듣습니다. 문자임을 알리려면 문자 양옆으로 큰따옴표(`""`)나 작은따옴표(`''`)를 붙여 줍니다. 소설 같은 문학작품에서 등장인물이 말할 때 큰따옴표를 붙이듯이 말이죠. 그러면 앞에서 오류가 난 부분에 적용해 봅시다.

```
print('가나다')
```

```
실행결과
가나다
```

이제 제대로 나옵니다. 문자를 표시할 때 큰따옴표와 작은따옴표 중 어느 것을 써도 상관없습니다. 이 책에서는 작은따옴표를 사용하겠습니다.

숫자에 작은따옴표를 붙이면 어떨까요? 다음을 실행해 봅시다.

```
print('123')
```

```
실행결과
123
```

오류 없이 123이 출력됩니다. `print(123)`과 `print('123')`은 결과가 같네요. `print(123)`은 숫자 123을 출력한 것이 맞습니다. 그런데 `print('123')`도 숫자 123을 출력한 것일까요?

`print(123)`과 `print('123')`의 결과가 같아 보이지만, 사실은 같지 않습니다. `print('123')`은 숫자 옆에 작은따옴표가 있으므로 컴퓨터는 숫자 123이 아닌 문자 123으로 이해합니다.

다음 코드를 실행해 보세요.

```
print(123 + 2)
print('123' + 2)
```

```
실행결과
125
---------------------------------------------------------------------------
      1 print(123 + 2)
----> 2 print('123' + 2)
TypeError: must be str, not int
```

`print(123 + 2)`는 123 + 2의 연산 결과인 125를 출력합니다. 그런데 `print('123' + 2)`는 오류가 발생합니다. 우리가 숫자 123(백이십삼)과 문자 백이십삼을 구분하는 것처럼 컴퓨터 역시 123과 '123'을 다르게 받아들입니다. 컴퓨터는 다음처럼 명령어를 이해했다고 볼 수 있습니다.

```
print('일이삼' + 2)
```

사람도 문자와 숫자는 더할 수 없습니다. 컴퓨터도 '명령어가 이상한데? 알아듣지 못해서 실행할 수 없어!'라며 오류 메시지를 보여 주는 것입니다.

2.2.3 사칙연산 출력하기

이번에는 다양한 형태의 연산을 `print()` 명령어에 넣어 실행해 봅시다.

```
print(123 + 9)
print(123 - 9)
```

```
실행결과
132
114
```

두 숫자를 덧셈하고 뺄셈한 결과가 나왔습니다. 다음은 어떨까요?

```
print('123 + 9')
print('123 - 9')
```

```
실행결과
123 + 9
123 - 9
```

소괄호 안의 수식이 계산되지 않고 그대로 출력됩니다. 이처럼 작은따옴표 안에는 수식을 넣어도 숫자가 아닌 문자로 인식합니다. 그래서 123 + 9와 123 - 9가 그대로 출력되죠. 컴퓨터가 문자 일이삼 + 구로 인식해 출력한 것입니다. `print(123)`과 `print('123')`이 같아 보이지만, 사실은 같지 않았죠. 마찬가지로 `print(123 + 9)`와 `print('123 + 9')`도 비슷해 보이지만 전혀 다릅니다. 마치 일란성 쌍둥이가 얼핏 보면 같아 보이지만, 사실은 전혀 다른 두 인물인 것처럼요.

덧셈과 나눗셈이 나왔으니 곱셈과 나눗셈도 해 볼까요? 108 × 8과 108 ÷ 8을 출력해 보세요.

> TIP 앗! 키보드에 덧셈과 뺄셈 기호는 있지만, 곱셈과 나눗셈 기호는 찾을 수 없네요. 어떻게 해야 할까요? 여러분도 알다시피 수학에서 사용하는 사칙연산의 기호는 +, -, ×, ÷입니다. 하지만 컴퓨터에서는 곱셈과 나눗셈 기호가 다릅니다. 컴퓨터에서는 ×가 알파벳의 x와 헷갈릴 수 있어서 `*`로 표시합니다. ÷도 컴퓨터에서 표현하기가 어려워서 ÷ 대신 /를 사용합니다. 다시 108 × 8과 108 ÷ 8을 출력해 보세요. 이번에는 출력할 수 있겠죠?

```
print(108 * 8)
print(108 / 8)
```

실행결과

```
864
13.5
```

사칙연산이 모두 잘 수행됩니다. 프로그래밍에서는 이런 사칙연산 기호를 연산자라고 합니다. 사칙연산 기호 외에도 프로그래밍에서 사용하는 연산자가 더 있습니다. 그중 몇 가지를 배워 보겠습니다.

2.2.4 기타 연산자 출력하기

프로그래밍하다 보면 나눗셈의 몫과 나머지가 필요할 때가 종종 있습니다. 앞에서 `print(108 / 8)`을 출력했을 때 결과는 몫과 나머지를 구분하지 않고 13.5로 나왔습니다. 여기에서 몫만 알고 싶다면 나눗셈 연산자를 2개 붙인 `//`를 사용하면 됩니다.

```
print(108 // 8)
```

```
실행결과
13
```

그럼 나머지만 알고 싶을 때 어떻게 할까요? 나머지를 구하는 연산자는 `%`입니다. 나눗셈 기호(`/`) 양 옆에 동그라미가 달린 형태로 퍼센트 기호라고 합니다. 프로그래밍에서 `%`는 백분율을 나타내는 단위인 퍼센트가 아니라 나머지를 나타내는 기호입니다.

```
print(108 % 8)
```

```
실행결과
4
```

이번에는 조금 더 재미있는 연산자를 배워 보겠습니다. 나눗셈 연산자가 2개면 몫을 구하는 연산자가 되는데, 곱셈 연산자가 2개면 어떻게 될까요?

```
print(108 ** 8)
```

```
실행결과
18509302102818816
```

엄청 큰 숫자가 나왔네요. 알아보기 쉽게 작은 숫자를 넣어 볼까요?

```
print(5 ** 2)
```

```
실행결과
25
```

곱셈 연산자를 두 번 쓰면 거듭제곱 연산자가 됩니다. `**` 연산자 앞의 숫자인 5가 밑이 되고, `**` 뒤의 숫자 2가 지수가 되어 5^2을 계산하는 것이죠. 따라서 앞에 나온 `print(108 ** 8)`은 108을 여덟제곱한(108^8) 값임을 알 수 있습니다.

지금까지 배운 연산자를 표로 정리하면 다음과 같습니다.

▼ 표 2-1 연산자

연산자	의미
`a + b`	a와 b를 더한 값
`a - b`	a에서 b를 뺀 값
`a * b`	a와 b를 곱한 값
`a / b`	a를 b로 나눈 값
`a // b`	a를 b로 나눈 몫
`a % b`	a를 b를 나눈 나머지
`a ** b`	a의 b제곱

2.2.5 여러 요소 연결해 출력하기

앞에서 숫자와 숫자를 연산자로 연결하면 숫자끼리 연산한 값이 출력됐습니다.

```
print(14 + 31)
```

```
실행결과
45
```

그럼 문자를 연산자로 연결하면 어떻게 출력될까요?

```
print('파이썬' + '프로그래밍')
```

```
실행결과
파이썬프로그래밍
```

`+` 연산자는 없어지고 두 단어가 한 단어처럼 붙어서 출력됩니다. 이처럼 문자와 문자에 `+` 연산자를 사용하면 문자를 연결해 출력할 수 있습니다.

문자를 연결해 출력하는 방법에는 `+` 연산자 외에도 콤마(`,`)를 사용하는 방법이 있습니다.

```
print('파이썬', '프로그래밍')
print(14, 31)
```

```
실행결과
파이썬 프로그래밍
14 31
```

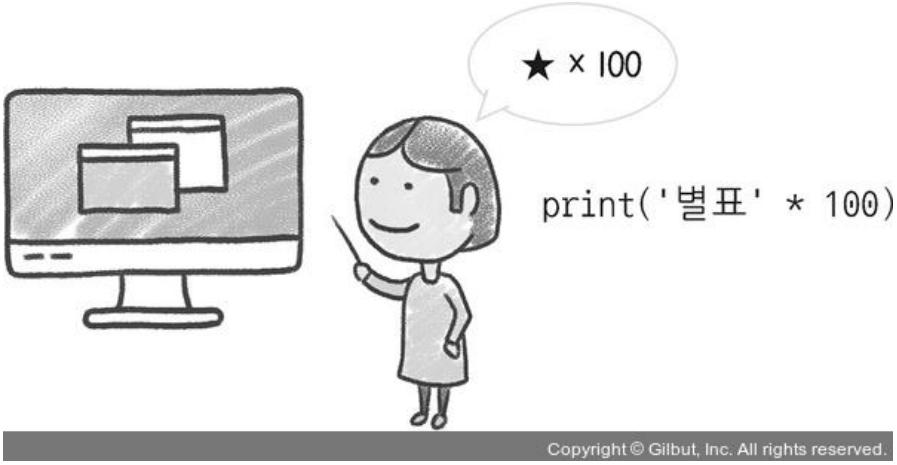
`+` 연산자와 마찬가지로 두 문자가 연결되어 출력됩니다. 단, `+`로 연결된 것과는 다르게 중간에 공백 한 칸이 생깁니다. 숫자도 연산 없이 문자처럼 연결되어 출력됩니다. 기호의 한글 이름(더하기표와 쉼표)을 생각해 보면 `+` 연산자와 콤마가 어떤 역할을 하는지 이해하기 쉬울 거예요. 콤마는 한 칸을 쉬고 그다음 문자를 출력하고, 더하기는 쉬지 않고 바로 다음 문자를 연결해 출력합니다.

`print(숫자 + 숫자)`는 두 숫자를 더해서 출력하고, `print(문자 + 문자)`는 두 문자를 연결해 출력합니다. 그럼 문자와 숫자에 `+` 연산자를 넣으면 어떨까요? 문자와 숫자로 사칙연산을 실행해 봅시다(책에서는 한 셀로 표시했지만, 코랩에서는 각각 셀에 넣어 실행해야 결과를 제대로 확인할 수 있습니다).

```
print('파이썬' + 3)
print('파이썬' - 3)
print('파이썬' / 3)
print('파이썬' * 3)
```

```
실행결과
----> 1 print('파이썬' + 3)
TypeError: must be str, not int
---------------------------------------------------------------------------
----> 1 print('파이썬' - 3)
TypeError: unsupported operand type(s) for -: 'str' and 'int'
---------------------------------------------------------------------------
----> 1 print('파이썬' / 3)
TypeError: unsupported operand type(s) for /: 'str' and 'int'
---------------------------------------------------------------------------
파이썬파이썬파이썬
```

`*` 연산자만 빼고 모두 오류가 발생합니다. 놀랍게도 문자에 숫자를 곱하면 오류가 발생하지 않고 곱한 숫자만큼 문자를 반복해 출력합니다. 수업 시간에 중요한 내용에 '별표 곱하기 100개'라는 말을 들은 적이 있지 않나요? 별 100개를 적는 것이 힘드니 간략하게 표현한 거죠. 이것도 비슷합니다. 문자와 숫자 연산에서 `*` 연산자는 문자를 숫자만큼 반복하라는 뜻입니다.



▲ 그림 2-2 문자와 숫자의 곱셈 연산

제제쌤의 조언

예외 사항이 많아 보일 수도 있습니다. 일일이 외우려 하지 마세요. 외워도 좋지만 외우지 못해도 필요할 때 찾아 쓸 수 있으면 됩니다. 오류가 났을 때 오류가 난 부분을 책에서 보고 고치거나 인터넷으로 오류 내용을 찾아서 이해할 수 있을 정도로만 익혀도 훌륭합니다. 책 내용을 전부 다 머릿속에 넣겠다고 생각하지 마세요. 일상생활을 예로 생각하면 굳이 외우지 않아도 자연스럽게 받아들일 수 있습니다. 최대한 자연스럽게 프로그래밍을 이해할 수 있게 하는 것이 이 책의 목표입니다!

다음은 어떨까요?

```
print('파이썬+프로그래밍')
print('파이썬 + 프로그래밍')
```

잠깐! 실행하기 전에 먼저 실행결과를 예상해 봅시다. 결과를 미리 예측하고 실제 결과와 비교하며 프로그래밍을 배운다면 훨씬 빠르고 재미있게 배울 수 있어요!

```
실행결과
파이썬+프로그래밍
파이썬 + 프로그래밍
```

`+` 연산자도 그대로 있고 작은따옴표 사이 문자는 공백까지 그대로 출력됩니다. 즉, 작은따옴표 사이에 있는 텍스트는 공백이든 숫자든 연산자든 한 단어처럼 인식해 그대로 출력됩니다.

지금까지 배운 내용 중에서 다른 궁금한 점이 있으면 얼마든지 시도해 보세요. 프로그래밍 실력은 많이 실행해 볼수록, 오류를 많이 내 볼수록 빠르게 성장합니다. 오류가 난다고 점수를 깎는 선생님이나 교수님은 없으니 마음껏 시도해 보세요!

1분 퀴즈

2 다음과 같이 출력하는 명령어는 무엇일까요?

> 실행결과
> 안녕하세요. 무엇이든 시켜 보세요.

① `안녕하세요. 무엇이든 시켜 보세요.`

② `print "안녕하세요. 무엇이든 시켜 보세요."`

③ `print '안녕하세요. + 무엇이든 시켜 보세요'.`

④ `print(안녕하세요. + 무엇이든 시켜 보세요.)`

⑤ `print('안녕하세요. 무엇이든 시켜 보세요.')`

정답 및 해설

2.3

데이터를 넣는 입력 명령어

지금까지 `print()`를 배웠습니다. `print()`는 컴퓨터가 괄호 안 내용을 실행한 결과를 보여 주는 출력 명령어였죠. 이번에는 출력 명령어와 반대되는 입력 명령어를 배워 보겠습니다. 입력 명령어를 배우기 전에 입력이 무엇인지부터 이해하고 넘어갈까요?

2.3.1 입력이란

프로그램을 사용하다 보면 다음 그림과 같은 빈칸을 많이 봅니다. SNS에 로그인할 때 아이디와 비밀번호를 넣어야 하는 빈칸, 인터넷 쇼핑 시 배송 받을 주소를 입력하는 빈칸, 포털 사이트 검색창의 빈칸 등 매우 많죠. 우리는 이 빈칸에 아이디, 비밀번호, 주소 등 요구하는 정보를 자연스럽게 넣죠. 이것이 바로 입력입니다. 프로그래밍에서 입력은 이처럼 사람이 직접 컴퓨터에 정보(데이터)를 넣어 주는 것을 의미합니다.



▲ 그림 2-3 실생활 속 다양한 입력 예시

프로그램의 결과를 사용자에게 보여 주는 것이 출력이라면, 입력은 사용자가 프로그램이 일하는 데 필요한 재료를 넣어 주는 것이라고 할 수 있습니다. 아이디와 비밀번호를 넣으면 로그인 화면에서는 입력된 아이디와 비밀번호로 사용자를 검색합니다. 도로명 주소를 입력하면 주소 데이터를 검색해 해당하는 우편번호를 출력합니다. 이처럼 프로그램이 수행되는 데 반드시 필요한 데이터를 사용자로부터 직접 받아야 하는 경우에 입력 명령어를 사용합니다.

2.3.2 입력 명령어 input( )

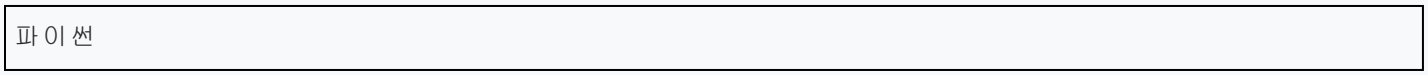
파이썬에서는 입력 명령어로 `input()`을 사용합니다. 앞서 배운 명령어 형식처럼 영단어와 소괄호가 조합된 형태입니다.

```
input(): 데이터를 입력받아라.
```

그럼 입력을 어떻게 받는지 확인해 봅시다. `input()`만 작성해서 실행합니다.

```
input()
```

실행결과



빈칸이 하나 나오고 실행 아이콘이 계속 돌아갑니다. 빈칸에 파이썬이라고 입력하고 키보드에서 Enter를 눌러 봅시다.

실행결과

```
파이썬
'파이썬'
```

입력한 단어가 결과창에 뜹니다. 컴퓨터가 데이터를 제대로 입력받았군요. 이번에는 `input()` 명령어를 다시 실행하고 다른 단어를 넣어 보세요. 그리고 입력한 데이터를 컴퓨터가 제대로 받아들이는지 확인해 보세요.

출력 명령어 `print()`에서 소괄호 안에 출력할 내용을 넣었죠? 입력 명령어 `input()`의 소괄호 안에도 내용을 넣을 수 있습니다. 먼저 숫자 3을 넣어 실행해 봅시다.

```
input(3)
```

실행결과

```
3 456
```

`3`이 출력되고 앞에서처럼 바로 옆에 빈칸이 나오고 실행 아이콘이 계속 돌아갑니다. 456을 입력하고 Enter를 눌러 보세요.

실행결과

```
3456
'456'
```

실행결과로 두 줄이 보입니다. 첫째 줄에는 `input()` 명령어 소괄호 안에 있는 `3`과 입력된 `456`이 연결되어 `3456`이 한 번에 보이죠. 둘째 줄에는 상자에 입력한 값인 `456`만 보입니다.

이번에는 문자를 넣어 보겠습니다. 파이썬에서는 문자 양옆에 반드시 작은따옴표가 있어야 하는 것을 유념하며 '입력'이라는 문자를 넣어 봅시다(참고로 다음 코드에는 입력 뒤에 띄어쓰기 한 칸이 있습니다).

```
input('입력 ')
```

실행결과

```
입력 7월 5일
```

숫자를 넣었을 때와 마찬가지로 괄호 안의 내용인 입력을 출력합니다. 그리고 바로 옆에 빈칸이 나옵니다. 이번에는 날짜를 넣고 Enter를 눌러 볼까요? 예시로 7월 5일을 입력해 보겠습니다.

> 실행결과
> 입력 7월 5일
> '7월 5일'

역시 첫째 줄에는 `input()` 소괄호 안의 내용과 입력한 내용이 같이 나타납니다. 둘째 줄은 입력한 내용만 나오고요. 두 예시로 짐작할 수 있겠지만, `input()`에서 소괄호 안에 내용이 있으면 해당 내용을 먼저 출력하고 바로 옆에 사용자에게서 입력을 받는 입력칸이 생깁니다. 그런데 이 기능이 왜 필요할까요? 예시를 변형해 `input()` 소괄호 안의 내용이 언제 필요한지 알아봅시다.

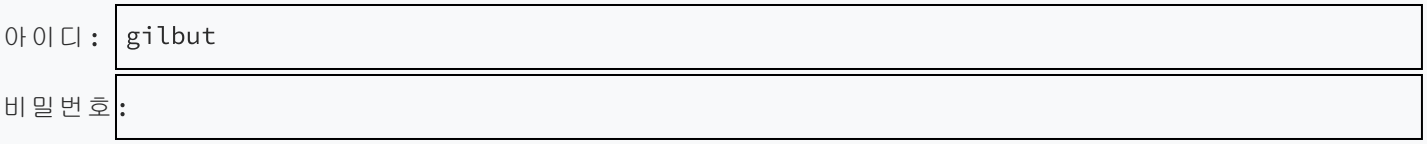
앞서 입력 개념을 설명할 때 예로 든 로그인 화면을 한 번 흉내내 볼까요? 다음과 같이 코드를 작성하고 실행합니다.

```
input('아이디: ')
input('비밀번호: ')
```

> 실행결과
> 아이디:

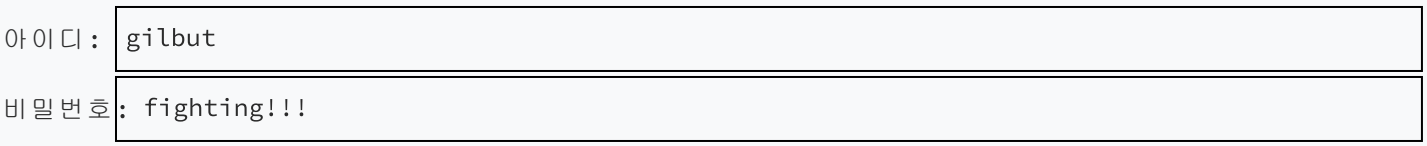
결과창이 로그인 화면과 비슷해 보이죠? 아이디에 gilbut을 넣고 Enter를 눌러 보세요.

실행결과



비밀번호를 입력받는 칸이 뜹니다. 이 역시 매우 익숙하죠. 비밀번호도 입력해 보겠습니다.

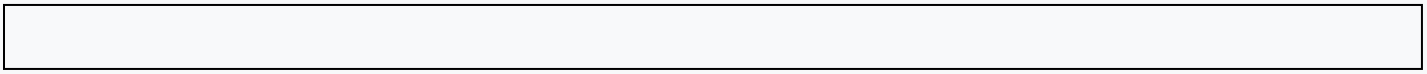
실행결과



2개의 데이터를 입력받은 로그인 화면을 흉내내 보았습니다. 그런데 방금 실행한 프로그램에서 `input()` 안에 텍스트가 없다면 어땠을까요?

```
input()
input()
```

실행결과



어떤 칸에 어떤 데이터를 입력해야 하는지 사용자가 알 수 없을 겁니다. 이것으로 `input()`에서 소괄호 안의 내용이 어떤 역할을 하는지 짐작할 수 있습니다. 소괄호의 내용은 입력할 내용을 설명하는 '안내문' 역할을 합니다. 예시에서는 첫째 칸에 아이디를, 둘째 칸에 비밀번호를 입력하라고 안내하는 역할입니다. `input('도로명 주소 입력: ')`이라고 하면 도로명 주소를 입력하라는 안내문이 되고 `input('학번을 입력하세요. --> ')`를 실행하면 오른쪽 빈칸에 학번을 입력하라는 안내문이 됩니다.

```
input('학번을 입력하세요. --> ')
```

실행결과

```
학번을 입력하세요. -->
```

TIP 코랩에서 입력 명령어를 실행하고 아무것도 입력하지 않으면 빈칸이 계속 보이고 실행 아이콘이 계속 돌아갑니다. 이 표시는 아직 명령어가 종료되지 않았음을 나타냅니다. 명령어를 아직 실행 중이라는 뜻이죠. 입력 명령어는 데이터가 입력되고 Enter를 눌러야 종료됩니다. 데이터를 입력하지 않으면 다음 코드를 실행할 수 없으니 유의하세요!

주피터 노트북도 마찬가지입니다. 입력 명령어를 실행하면 코드 칸 왼쪽에 In[*] 표시가 보입니다. 이 역시 명령어가 종료되지 않고 아직 실행 중이라는 뜻이죠. 데이터가 입력되고 Enter를 누르면 별표(*)가 숫자로 변하는 게 보일 거예요. 별표가 사라지지 않으면 다음 셀이 실행되지 않으니 꼭 입력 데이터를 넣은 후 Enter누르기를 잊지 마세요.



Copyright © Gilbut, Inc. All rights reserved.

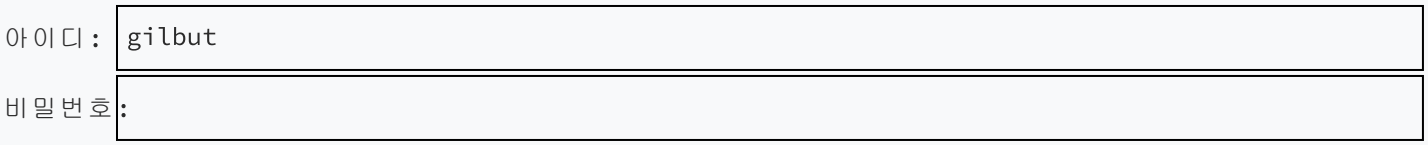
▲ 그림 2-4 input() 실행 시

2.3.3 명령어의 실행 순서

로그인 화면을 흉내낸 예제로 다시 가봅시다. 여기에는 간단하지만 중요한 개념 하나가 숨어 있습니다. 이 예제에서는 처음으로 명령어 1개가 아닌 2개를 동시에 실행해 봤습니다. 2개의 명령어 중 어떤 명령어가 먼저 실행됐나요? 맞습니다. 첫 줄에 아이디를 입력받는 명령어가 먼저 실행됐습니다. 아이디 입력칸에 데이터를 넣고 Enter를 누르고 나서야 밑에 있는 두 번째 명령어가 실행되어 비밀번호를 입력받는 칸이 나타나죠.

```
input('아이디: ')    ← 첫 번째 실행
input('비밀번호: ')  ← 두 번째 실행
```

실행결과



이번에는 두 명령어의 순서를 바꿔 실행해 볼까요?

```
input('비밀번호: ')  ← 첫 번째 실행
input('아이디: ')    ← 두 번째 실행
```

두 명령어의 순서를 바꿔도 역시 첫째 줄에 있는 명령어가 먼저 실행되어 비밀번호를 입력하라는 명령어가 실행됩니다. 우리가 아는 로그인 화면과는 조금 다르죠.

```
실행결과
비밀번호: fighting!!!
```

비밀번호를 입력하고 Enter 를 눌러 봅시다.

```
실행결과
비밀번호: fighting!!!
아이디 :
```

아이디를 입력받는 빈칸이 이제서야 나타납니다. 즉, 두 번째 명령어가 실행됐습니다.

정리해 보겠습니다. 여러 개의 명령어가 있을 때 실행되는 순서는 위에서 아래입니다. 명령어의 실행 순서는 매우 중요합니다. 명령어의 실행 순서대로 프로그램의 흐름을 설계하기 때문입니다. 프로그래밍에서 프로그램의 흐름 즉, 명령어의 실행 순서를 제어구조라고 합니다.

> TIP 갑자기 왜 제어구조가 나왔을까요? 앞에서 설명했듯이 제어구조는 명령어의 실행 순서입니다. 명령어가 하나일 때는 실행 순서를 신경 쓸 이유가 없습니다. 명령어 하나만 실행한 후 프로그램을 종료하면 되니까요. 하지만 명령어가 2개 이상일 때는 명령어 사이에 순서가 생기게 됩니다. 출력 명령어와 입력 명령어를 배우고 나면 여러 개의 명령어를 다루게 되므로 비교적 간단한 명령어를 배울 때 제어구조의 개념을 잡고 넘어가는 것이 프로그래밍을 이해하는 데 도움이 됩니다.

우리가 이 책에서 배울 제어구조는 총 3개입니다. 그중 가장 기본인 구조를 앞에서 다루었습니다. 바로 순차구조입니다. 순차구조는 차례대로(순서대로) 진행되는 구조를 의미합니다. 그럼 차례의 의미를 한번 생각해 봅시다. 위와 아래가 있을 때 위에서 아래로 가는 것이 순서일까요? 아래에서 위로 가는 것이 순서일까요? 위에서 아래로 진행되는 것이 일반적인 순서죠.



▲ 그림 2-5 순차구조

우리 주변에서 차례대로 진행되는 것은 무엇이 있을까요? 가장 흔한 것은 아무래도 책이 아닐까 생각이 드네요. 책을 읽을 때 윗줄부터 아랫줄로 읽어 내려가죠. 명령어 역시 책을 읽을 때처럼 윗줄의 명령어가 먼저 실행되고 아랫줄로 내려가면서 차례대로 실행됩니다.

2.3.4 여러 명령어 함께 실행하기

그렇다면 명령어 2개가 아니라 3개일 때는 어떨까요? 다음을 실행해 봅시다. 실행하기 전 먼저 결과를 예상해 보고 예상이 맞는지 스스로 확인하세요.

```
input('아이디: ')
input('비밀번호: ')
print('gilbu님 환영합니다! ')
```

```
실행결과
아이디: gilbut
비밀번호: fighting!!!
gilbut님 환영합니다!
```

예상대로 실행됐나요? 그리 어렵지 않았을 겁니다. 아이디와 비밀번호를 순서대로 입력받은 후, 마지막으로 환영인사가 출력됩니다. 하나만 더 해 볼까요? 이번에는 회원가입 페이지를 흉내낸 프로그램을 작성하겠습니다.

```
print ('---회원가입---')      ← 첫 번째 실행
input('아이디: ')             ← 두 번째 실행
print('가입을 환영합니다! ')  ← 세 번째 실행
```

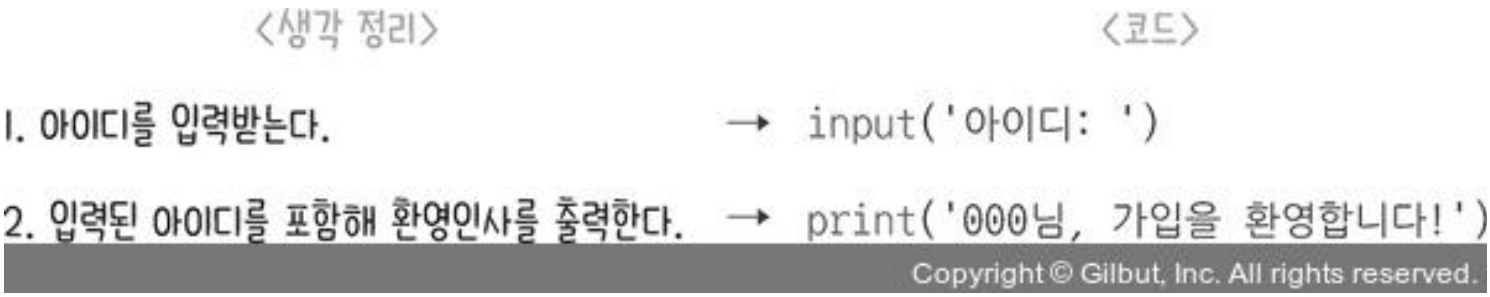
```
실행결과
---회원가입---
아이디: gilbut
가입을 환영합니다!
```

먼저 `'---회원가입---'` 문구가 출력된 후, 아이디를 입력하는 빈칸이 나타납니다. 아이디를 입력하고 나면 사이트 가입을 환영하는 메시지가 나오네요. 명령어의 내용이 무엇이든 위에서 아래로 진행되는 순차구조에 따라 프로그램이 실행됩니다. 한 프로그램 안에 명령어가 10개든 50개든 순차구조로 실행된다는 점을 꼭 알아 두세요!

앞의 프로그램 실행결과에서 마지막 줄을 봅시다. '가입을 환영합니다!'보다는 'gilbut님, 가입을 환영합니다!'처럼 회원 아이디를 넣어서 출력하면 회원을 더욱더 신경 쓴 느낌이 날 것 같습니다. 어떻게 해야 할까요? 원하는 결과는 다음과 같습니다.

> 원하는 실행결과
>
> 아이디: gilbut
> ---회원가입---
> 아이디: gilbut
> gilbut님! 가입을 환영합니다!

원하는 결과가 명확하니 결과를 중심으로 어떻게 명령어를 작성할지 생각해 봅시다. 먼저 아이디를 입력받아야 합니다. 아이디를 입력받는 부분은 이미 알고 있는 것처럼 `input('아이디: ')`면 되죠. 그럼 'gilbut님, 가입을 환영합니다!'는 어떻게 작성해야 할까요? 아이디(gilbut) 부분을 제외하고 생각하면 문자로 '님, 가입을 환영합니다!'를 출력하면 됩니다. 이 역시 앞에서 배운 부분이라 쉬울 겁니다. `print('님, 가입을 환영합니다!')`를 실행하면 되죠. 이제 아이디만 남았네요. 우선 지금까지 완성된 부분을 간단하게 정리하고, 코드로 표현해 봅시다.



Copyright © Gilbut, Inc. All rights reserved.

할일 순서를 작성하듯이 생각을 정리한 후 프로그래밍하면 더욱 효율적으로 접근할 수 있습니다. 순서에 맞는 명령어를 대응시키면 되기 때문입니다. 앞 그림을 보면 <생각 정리>에서 프로그램을 두 단계로 나누어서 정리했고, <코드>에서 각 단계를 적절한 명령어로 표현했습니다.

두 번째 줄의 `000` 부분을 어떻게 해결할 수 있을까요? 입력된 아이디가 출력되길 원하니 첫 번째 줄에서 `input()`으로 아이디를 입력받는 코드를 그대로 넣어 주면 됩니다.

그리고 한 가지 더 추가해야 합니다. 현재 `print()` 안에 2개의 요소(`input()`과 문자)가 있기 때문에 두 요소를 연결해서 출력해야 합니다. 여러 문자를 연결해 한 번에 출력할 때 어떻게 했죠? `+` 연산자 또는 콤마(`,`)를 사용했습니다. 마찬가지로 명령어와 문자를 연결할 때도 `+` 연산자 또는 콤마(`,`)를 사용해야 합니다. 여기서는 `+` 연산자를 사용하겠습니다.

```
print(input('아이디: ') + '님, 가입을 환영합니다!')
```

실행결과

```
아이디:
```

아이디를 입력하는 빈칸이 나타납니다. 아이디를 입력하고 Enter를 눌러보세요.

실행결과

```
아이디: gilbut
gilbut님, 가입을 환영합니다!
```

아이디를 입력하니 원하던 결과가 출력됩니다. 명령어 안에 명령어를 사용했는데 이것이 어떻게 가능할까요? 더 이해하기 쉽게 익숙한 수학 연산을 예로 들어 보겠습니다.

사칙연산의 순서는 곱하기 = 나누기 > 더하기 = 빼기입니다. 그리고 곱하기와 나누기보다 더 먼저 계산되는 것이 바로 괄호입니다. 실제 수식으로 봅시다.

((3 + 7) - 2) / 10

괄호 안에 괄호가 또 있네요. 이럴 경우 알다시피 가장 안쪽 괄호를 먼저 계산한 후, 바깥 괄호를 계산하는 것이 원칙입니다. 가장 안쪽 괄호 (3 + 7)은 10이니 (3 + 7)을 10으로 대체해 보겠습니다.

(10 - 2) / 10

괄호가 여전히 하나 남았네요. 만약 괄호가 없다면 나눗셈이 뺄셈보다 먼저 계산되지만, 지금은 뺄셈이 괄호 안에 있으니 (10 - 2)를 먼저 계산합니다. (10 - 2)는 8이 되고, 다시 수식에 반영하면 8 / 10이 되므로 최종 결과는 0.8입니다.

앞에서 작성한 명령어를 다시 볼까요? `print()` 명령어 안에 `input()` 명령어가 있는 형태지요. 이럴 때는 `print()`의 괄호 안에 있는 `input()` 명령어가 먼저 실행되고 그 후에 바깥에 있는 출력 명령어가 실행됩니다. 여러 개의 괄호가 있는 수식에서 가장 안쪽 괄호를 먼저 계산한 후에 바깥쪽 괄호를 계산하는 것처럼요.



명령어가 여러 개일 때 위에서 아래로 나열해 여러 줄로 표현할 수도 있지만, 명령어 안에 명령어를 넣어 작성할 수도 있습니다. 이처럼 명령어를 겹쳐 쓰는 형태는 프로그래밍이 익숙해질수록 자주 사용하게 됩니다. 전체 프로그램의 줄 수를 줄일 수 있기 때문인데요. 2줄의 코드를 1줄로 줄일 수 있고, 더 전문적인 프로그램에서는 4~5줄의 코드를 1줄로 줄일 수도 있습니다.

그럼 회원가입 코드를 완성해 볼까요?

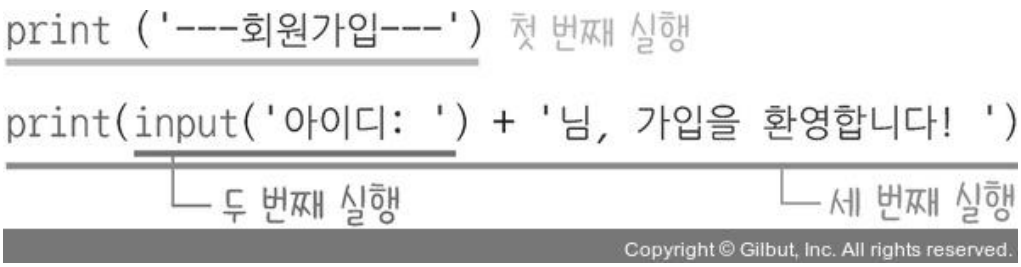
```
print ('---회원가입---')
print(input('아이디: ') + '님, 가입을 환영합니다! ')
```

실행결과

```
---회원가입---
아이디: gilbut
gilbut님, 가입을 환영합니다!
```

비록 진짜로 회원가입이 되는 프로그램은 아니었지만, 그래도 그럴싸해 보이네요.

마지막으로 이 프로그램의 실행 순서를 한번 볼까요? 명령어가 여러 줄이 있으므로 위부터 아래로 진행되어 첫째 줄의 `print()` 명령어가 먼저 실행됩니다. 그다음으로 둘째 줄이 실행됩니다. 그런데 명령어 안에 명령어가 있으므로 가장 안쪽에 있는 `input()` 명령어가 두 번째로 실행되고, 마지막으로 바깥쪽 명령어인 `print()`가 실행됩니다.



지금까지 `input()` 명령어를 살펴보았습니다. `input()` 명령어는 다음 장에서 변수의 개념을 배우면서 더 다양하게 다뤄 보겠습니다. 이 장에서는 `input()` 명령어의 형식과 여러 개의 명령어가 있을 때 프로그램의 실행 흐름(제어구조)을 중점적으로 봐 주세요.

1분 퀴즈

3 다음 중 틀린 것을 고르세요.

① 아이디와 비밀번호를 입력받는 로그인 화면이 있을 때, 필요한 입력 명령어의 개수는 2개다.

② 입력 명령어는 `input()`이며, `input()`을 실행하면 키보드로 직접 값을 넣을 수 있다.

③ `input()` 명령어의 소괄호 안에 적절한 텍스트를 넣으면 입력을 안내하는 문구로 활용할 수 있다.

④ 명령어 안에 또 다른 명령어를 넣어 두 명령어를 한 줄의 코드로 실행할 수 없다.

⑤ 명령어가 여러 개 있을 때, 위에서부터 아래에 있는 명령어순으로 실행된다.

정답 및 해설

2

마무리

이 장에서 배운 내용을 정리해 보겠습니다.

1 출력 명령어

명령어	실행결과	설명
`print()`		출력 명령어
`print(5)`	`5`	숫자 출력
`print('파이썬')` 또는 `print("파이썬")`	`파이썬`	문자 출력
`print(5 ** 2)`	`25`	수식을 넣으면 연산 결과 출력

2 여러 요소 연결해서 출력하기

명령어	실행결과	설명
`print('문자' + '문자')` 예 `print('온라인' + '쇼핑')`	`온라인쇼핑`	2개의 문자를 공백 없이 이어서 출력

`print('문자', '문자')`
`print(숫자, 숫자)`
예 `print('온라인', '쇼핑')`
예 `print(20, 20)`
	`온라인 쇼핑`
	`20 20`
		공백을 포함해 2개의 요소를 이어서 출력

`print('문자' * 숫자)` 예 `print('블링' * 2)`	`블링블링`	문자를 숫자 횟수만큼 출력

3 입력 명령어

명령어	실행결과	설명
`input()`	입력칸만 생성	입력 명령어
`input('입력 안내문')`	입력 안내문을 출력 후 바로 입력칸 생성	입력 안내문을 포함한 입력 명령어

4 명령어 실행 순서(제어구조)

명령어	실행결과
명령어가 여러 줄일 때	위에서부터 아래로 실행
명령어 안에 명령어가 있을 때	가장 안쪽 명령어부터 바깥쪽 명령어 순서로 실행

Self Check

1 다음은 [무궁화 꽃이 피었습니다] 놀이를 진행할 때 등장하는 말입니다. 다음 결과처럼 출력되도록 `print()` 명령어로 프로그램을 작성하세요(단, 하나의 `print()` 명령어 안에 문자와 연산자를 적절히 조합하세요).

> 실행결과
> 무궁화 꽃이 피었습니다! 무궁화 꽃이 피었습니다! 무궁화 꽃이 피었습니다!

2 다음 코드를 실행한 결과를 예상해 보세요. 정상적으로 실행된다면 결괏값을 적어 보고, 그렇지 않다면 그 이유를 설명해 보세요.

```
print('95' + 3)
print('95' * 3)
```

3 다음 결과처럼 출력되도록 닉네임을 입력받고 환영 인사를 해 보세요. `print()`와 `input()`을 적절히 활용하세요.

> 실행결과
> 닉네임을 입력해 주세요. --> 라이언
> 라이언님 안녕하세요.

힌트 출력 결과를 두 부분으로 나누어 생각해 보세요!

정답 및 해설